

INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo (ESCOM / UPIIZ)

Ingeniería en Sistemas Computacionales

TEMA

**Números especiales en Kotlin:
primo, Fibonacci, número maravilloso
(conjetura de Collatz) y constante de Kaprekar**

Asignatura: Desarrollo de Aplicaciones Móviles Nativas

Práctica: Práctica 4: Operadores y estructuras de control de Kotlin

Alumno: Angel Frausto Robles

Grupo: 7CM4

Fecha: 12 de septiembre de 2026

INTRODUCCIÓN

Esta práctica corresponde al tema "Operadores y estructuras de control de Kotlin" de la Unidad II del programa de la materia. Parte del ejemplo del profesor que verifica si un número natural es primo, y lo extiende con dos ejercicios: comprobar si el número es de Fibonacci y si es un "número maravilloso" (la conjetura de Collatz), y comprobar si un número de 3 o 4 dígitos corresponde a la constante de Kaprekar.

Objetivos:

- Aplicar operadores de rango, comparación y el operador Elvis (`?:`) de Kotlin en un caso real.
- Practicar estructuras de control (`for` , `while` , `when`) para resolver los cuatro criterios.
- Construir una interfaz Android simple (EditText, Button, TextView) que reciba un número y muestre el resultado de los cuatro análisis.
- Verificar el funcionamiento de la aplicación en un emulador Android con distintos casos de prueba.

DESARROLLO

1. Entorno de trabajo

Se creó un proyecto en Android Studio con la plantilla *Empty Views Activity* y lenguaje Kotlin. El proyecto usa Android Gradle Plugin 9.3.2 (soporte de Kotlin integrado, sin plugin adicional de `org.jetbrains.kotlin.android`), `compileSdk / targetSdk` 37 y `minSdk` 24. La actividad principal es `MainActivity`, que extiende `AppCompatActivity`.

2. Interfaz (activity_main.xml)

La interfaz se armó con un `ScrollView` que contiene un `LinearLayout` vertical con tres elementos: un `EditText` para capturar el número natural, un `Button` ("Verificar") y un `TextView` donde se muestra el resultado de los cuatro análisis. Se usó `ScrollView` porque la secuencia de Collatz de algunos números puede ocupar varias líneas de texto.

```
<ScrollView ...>
    <LinearLayout android:orientation="vertical" ...>
        <EditText android:id="@+id/xnatural" android:inputType="number"
            android:hint="@string/hint_numero" />
        <Button android:id="@+id/xboton" android:text="@string/accion_verificar" />
        <TextView android:id="@+id/xcalcular" android:fontFamily="monospace" />
    </LinearLayout>
</ScrollView>
```

3. Lógica de los cuatro análisis (MainActivity.kt)

La función `esPrimo` descarta los números menores o iguales a 1 y busca divisores hasta la raíz cuadrada del número:

```
private fun esPrimo(num: Int): Boolean {
    if (num <= 1) return false
    for (i in 2..sqrt(num.toDouble()).toInt()) {
        if (num % i == 0) return false
    }
    return true
}
```

Para el Ejercicio 1, `esFibonacci` genera la sucesión 0, 1, 1, 2, 3, 5... hasta alcanzar o pasar el número; es de Fibonacci si la sucesión cae exactamente en él:

```
private fun esFibonacci(num: Int): Boolean {
    var a = 0
    var b = 1
    while (a < num) {
        val siguiente = a + b
        a = b
        b = siguiente
    }
    return a == num
}
```

El número maravilloso, también del Ejercicio 1, sigue la conjetura de Collatz: si el número es par se divide entre 2, si es impar se multiplica por 3 y se suma 1, y se repite hasta llegar a 1. `secuenciaMaravilloso` usa `Long` porque $3n+1$ puede exceder el rango de `Int`:

```
private fun secuenciaMaravilloso(num: Int): List<Long> {
    val secuencia = mutableListOf(num.toLong())
    var n = num.toLong()
    while (n != 1L && secuencia.size < 1000) {
        n = if (n % 2 == 0L) n / 2 else 3 * n + 1
        secuencia.add(n)
    }
    return secuencia
}
```

El Ejercicio 2 pide la constante de Kaprekar, que solo aplica a números de 3 o 4 dígitos. En cada paso, `pasosKaprekar` ordena los dígitos de forma ascendente y descendente y resta el menor del mayor; la rutina converge a 495 (3 dígitos) o 6174 (4 dígitos), salvo que los dígitos sean todos iguales:

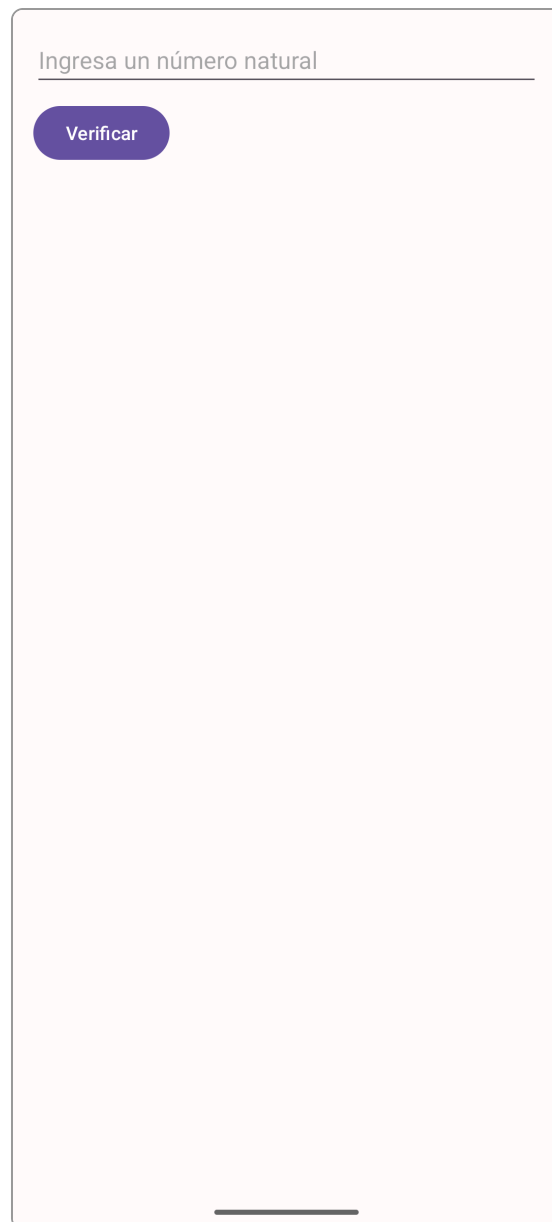
```
private fun pasosKaprekar(num: Int): List<Int>? {
    val digitos = num.toString().length
    if (digitos !in 3..4) return null
    val objetivo = if (digitos == 3) 495 else 6174
    val pasos = mutableListOf(num)
    var n = num
    while (n != objetivo && n != 0 && pasos.size < 20) {
        val ascendente = n.toString().padStart(digitos, '0').toList().sorted().joinToString("")
        val descendente = ascendente.reversed()
        n = descendente.toInt() - ascendente.toInt()
        pasos.add(n)
    }
    return pasos
}
```

Al presionar el botón "Verificar", se valida la entrada con `toIntOrNull() ?: -1` (operador Elvis) y, si es válida, se arma el resultado con `buildString`, evaluando los cuatro criterios y usando un bloque `when` para el caso de Kaprekar.

4. Pruebas en el emulador Android

La aplicación se instaló y ejecutó en un emulador Android (Medium Phone, API 36) con `adb`. Se probaron los siguientes casos, que en conjunto cubren los cuatro análisis:

Número	Primo	Fibonacci	Maravilloso	Kaprekar
13	Sí	Sí	Sí (9 pasos)	No aplica (2 dígitos)
495	No	No	Sí (97 pasos)	Sí, ya es la constante (495, 0 pasos)
2025	No	No	Sí (156 pasos)	Sí, llega a 6174 en 6 pasos



Ingresa un número natural

Verificar

The image shows a mobile application interface with a light pink background. At the top, there is a text input field with the placeholder text "Ingresa un número natural". Below the input field is a purple button with the text "Verificar". The bottom of the screen features a black horizontal line, representing the home indicator on an iPhone.

Figura 1. Pantalla inicial de la aplicación.

13

Verificar

Número analizado: 13

[SÍ] Primo
[SÍ] Fibonacci
[SÍ] Maravilloso (Collatz)
13 -> 40 -> 20 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
pasos: 9

Kaprekar: no aplica (requiere 3 o 4 dígitos)

Figura 2. Número 13: primo, Fibonacci y maravilloso.

495

Verificar

Número analizado: 495

[NO] Primo
[NO] Fibonacci
[SÍ] Maravilloso (Collatz)
495 -> 1486 -> 743 -> 2230 -> 1115 -> 3346 -> 1673 -> 5020 -> 2510 -> 1255 -> 3766 -> 1883 -> 5650 -> 2825 -> 8476 -> 4238 -> 2119 -> 6358 -> 3179 -> 9538 -> 4769 -> 14308 -> 7154 -> 3577 -> 10732 -> 5366 -> 2683 -> 8050 -> 4025 -> 12076 -> 6038 -> 3019 -> 9058 -> 4529 -> 13588 -> 6794 -> 3397 -> 10192 -> 5096 -> 2548 -> 1274 -> 637 -> 1912 -> 956 -> 478 -> 239 -> 718 -> 359 -> 1078 -> 539 -> 1618 -> 809 -> 2428 -> 1214 -> 607 -> 1822 -> 911 -> 2734 -> 1367 -> 4102 -> 2051 -> 6154 -> 3077 -> 9232 -> 4616 -> 2308 -> 1154 -> 577 -> 1732 -> 866 -> 433 -> 1300 -> 650 -> 325 -> 976 -> 488 -> 244 -> 122 -> 61 -> 184 -> 92 -> 46 -> 23 -> 70 -> 35 -> 106 -> 53 -> 160 -> 80 -> 40 -> 20 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
pasos: 97

Kaprekar: Sí llega a la constante 495
pasos: 0

Figura 3. Número 495: constante de Kaprekar (3 dígitos, 0 pasos).

2025

Verificar

Número analizado: 2025

[NO] Primo
[NO] Fibonacci
[SÍ] Maravilloso (Collatz)
2025 -> 6076 -> 3038 -> 1519 -> 4558 -> 2279 -> 6838 -> 3419 -> 10258 -> 5129 -> 15388 -> 7694 -> 3847 -> 11542 -> 5771 -> 17314 -> 8657 -> 25972 -> 12986 -> 6493 -> 19480 -> 9740 -> 4870 -> 2435 -> 7306 -> 3653 -> 10960 -> 5480 -> 2740 -> 1370 -> 685 -> 2056 -> 1028 -> 514 -> 257 -> 772 -> 386 -> 193 -> 580 -> 290 -> 145 -> 436 -> 218 -> 109 -> 328 -> 164 -> 82 -> 41 -> 124 -> 62 -> 31 -> 94 -> 47 -> 142 -> 71 -> 214 -> 107 -> 322 -> 161 -> 484 -> 242 -> 121 -> 364 -> 182 -> 91 -> 274 -> 137 -> 412 -> 206 -> 103 -> 310 -> 155 -> 466 -> 233 -> 700 -> 350 -> 175 -> 526 -> 263 -> 790 -> 395 -> 1186 -> 593 -> 1780 -> 890 -> 445 -> 1336 -> 668 -> 334 -> 167 -> 502 -> 251 -> 754 -> 377 -> 1132 -> 566 -> 283 -> 850 -> 425 -> 1276 -> 638 -> 319 -> 958 -> 479 -> 1438 -> 719 -> 2158 -> 1079 -> 3238 -> 1619 -> 4858 -> 2429 -> 7288 -> 3644 -> 1822 -> 911 -> 2734 -> 1367 -> 4102 -> 2051 -> 6154 -> 3077 -> 9232 -> 4616 -> 2308 -> 1154 -> 577 -> 1732 -> 866 -> 433 -> 1300 -> 650 -> 325 -> 976 -> 488 -> 244 -> 122 -> 61 -> 184 -> 92 -> 46 -> 23 -> 70 -> 35 -> 106 -> 53 -> 160 -> 80 -> 40 -> 20 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
pasos: 156

Kaprekar: Sí llega a la constante 6174
2025 -> 4995 -> 5355 -> 1998 -> 8082 -> 8532 -> 6174
pasos: 6

Figura 4. Número 2025: constante de Kaprekar (4 dígitos, 6 pasos).

CONCLUSIONES

Esta práctica me sirvió para reforzar cómo se combinan las estructuras de control de Kotlin (`for` , `while` , `when`) con operadores propios del lenguaje, como el Elvis (`?:`) para manejar entradas inválidas sin necesidad de bloques `if` adicionales. El ejercicio del número maravilloso me pareció el más interesante: la lógica es muy corta, una condición de paridad y dos operaciones, pero la secuencia resultante para números como 495 o 2025 puede tardar decenas de pasos en llegar a 1. Es la conjetura de Collatz: nadie ha demostrado que funcione para todos los números naturales, aunque se ha comprobado con computadoras para cifras enormes.

La rutina de Kaprekar también deja una idea clara de por qué a veces conviene expresar un cálculo como una secuencia de pasos (una lista) en lugar de solo un resultado final: mostrar los pasos intermedios ayuda a verificar que el algoritmo hace lo que se espera, y en este caso corrobora visualmente que la rutina siempre converge a 495 o 6174 salvo en el caso especial de dígitos repetidos.

BIBLIOGRAFÍA

- Cifuentes A., Alejandro. *DAMN Kotlin Practica4* (documento de la materia, Desarrollo de Aplicaciones Móviles Nativas).
- Kotlin Foundation. *Kotlin documentation: Control flow, Operator overloading, Null safety*.
<https://kotlinlang.org/docs/>
- Android Developers. *Build your first app / App architecture guides*. <https://developer.android.com/guide>
- Wikipedia. *Collatz conjecture*. https://en.wikipedia.org/wiki/Collatz_conjecture
- Wikipedia. *Kaprekar's constant*. [https://en.wikipedia.org/wiki/6174_\(number\)](https://en.wikipedia.org/wiki/6174_(number))